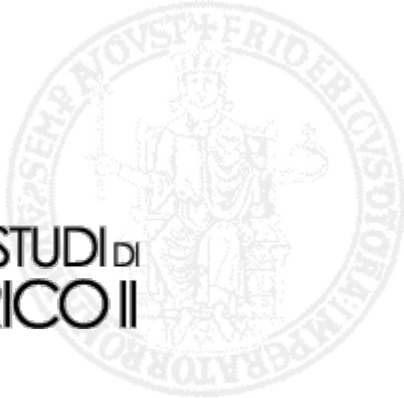




UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Cloud Platform And Infrastructure as Code

*Scalamarket: Using API, Kubernetes
and AWS for developing a
Microservice-based E-Commerce
Platform*

De Lucia Simone M63001720

Indice

1	Presentazione del Progetto	1
1.1	Descrizione Generale	1
1.2	Requisiti dell'Applicativo	1
1.3	Il Sito Web implementato con Flask	2
2	Documentazione Endpoints API	3



Capitolo 1

Presentazione del Progetto

1.1 Descrizione Generale

Scalamarket è una moderna piattaforma e-commerce basata su un'architettura a microservizi, progettata per garantire elevata scalabilità, affidabilità e flessibilità. Il sistema gestisce l'intero ciclo di vita dell'acquisto online: dalla registrazione degli utenti, alla consultazione del catalogo, fino all'elaborazione degli ordini e alla gestione logistica delle spedizioni. L'intera infrastruttura è stata ideata per essere distribuita in modo efficiente tramite Kubernetes ed eventualmente ospitata in ambiente cloud AWS, fornendo un backend solido, scalabile e supportato da API RESTful.

TODO: Inserire qui un diagramma ad alto livello dell'architettura di Scalamarket (Microservizi, Frontend, Database).

1.2 Requisiti dell'Applicativo

L'applicativo è stato sviluppato per soddisfare una serie di requisiti architetturali, funzionali e di business specifici, documentati nei requisiti di progetto:

- **Autenticazione e Autorizzazione (Auth Service):** Il sistema deve permettere la registrazione e il login sicuri sia per i clienti (Client) che per gli amministratori (Admin). L'amministratore deve avere i privilegi per gestire le utenze.
- **Gestione Inventario (Inventory Service):** Il sistema deve gestire un catalogo dinamico di articoli, tenendo traccia delle quantità ripartite in molteplici filiali. Deve inoltre impedire la creazione di ordini per articoli non disponibili.
- **Gestione Ordini e Logistica (Order Service):** L'applicativo deve processare il carrello dell'utente, allocando in tempo reale i prodotti necessari e scalandoli in modo intelligente dalle filiali che ne



hanno disponibilità. Deve inoltre tracciare l'occupazione dei mezzi logistici, gestendo i furgoni in uso e quelli a disposizione.

- **Architettura a Microservizi e Dati Isolati:** Per massimizzare la modularità, ogni servizio deve possedere una propria logica di business e interfacciarsi con il proprio database PostgreSQL indipendente, disaccoppiando così i domini applicativi.

TODO: Inserire qui un diagramma Entità-Relazione (ER) per i database o un flowchart della logica di allocazione degli ordini.

1.3 Il Sito Web implementato con Flask

Il lato client e il pannello di gestione amministrativa sono stati realizzati mediante un'interfaccia web responsive integrata con il backend sviluppato in Flask. L'applicazione Flask espone e gestisce tutti gli endpoint necessari, mentre la UI comunica costantemente con questi servizi tramite chiamate asincrone.

- **Interfaccia Cliente:** L'utente può scorrere l'intero catalogo, visualizzare i prezzi e le disponibilità massime, aggiungere elementi al carrello e, una volta inviato l'ordine, monitorarne lo stato e i dettagli dei prodotti tramite un'apposita cronologia.
- **Interfaccia Amministratore:** L'admin ha accesso a una dashboard completa che gli permette di gestire l'inventario (con possibilità di ricerca rapida tramite nome e identificativo), controllare in tempo reale lo stato logistico (veicoli in uso, totali e disponibili), visionare lo storico ordini globale e monitorare/eliminare gli account utenti.

TODO: Inserire qui degli screenshot rappresentativi delle schermate del sito web (es. Dashboard Utente, Pannello di Controllo Admin).



Capitolo 2

Documentazione Endpoints API

Questo documento presenta tutti gli endpoint implementati nell'applicativo Scalamarket. Il sistema è basato su un'architettura a microservizi, composta da: Auth Service (Porta 5001), Inventory Service (Porta 5002) e Order Service (Porta 5003). Di seguito, le tabelle sono suddivise in base ai file di routing (es. `client.py` e `admin.py`) che implementano i vari endpoint.

Auth Service

/client/register			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Registra un nuovo utente cliente nel sistema.	{username, password}	201 (Created) 400 (Bad Request)
/client/login			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Autentica un cliente e ne restituisce le informazioni.	{username, password}	200 (OK) 401 (Unauthorized)

Tabella 2.1: Auth Service - `routes/client.py`



/admin/register			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Registra un nuovo amministratore.	{username, password}	201 (Created) 400 (Bad Request)
/admin/login			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Autentica un amministratore e ne restituisce i dati.	{username, password}	200 (OK) 401 (Unauthorized)
/admin/users			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Restituisce la lista di tutti gli utenti registrati.	Nessuno	200 (OK)
/admin/users/<id>			
	Descrizione	Parametri Richiesti	Codici HTTP
DELETE	Elimina definitivamente l'utente specificato dall'ID.	Nessuno (id nell'URL)	200 (OK) 404 (Not Found)

Tabella 2.2: Auth Service - routes/admin.py



Inventory Service

/client/catalog			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Ottiene il catalogo completo degli articoli disponibili per i clienti.	Nessuno	200 (OK)

Tabella 2.3: Inventory Service - routes/client.py

/admin/article			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Aggiunge un nuovo articolo all'inventario con relative giacenze.	{name, description, price, quantities}	201 (Created) 400 (Bad Request)
/admin/inventory			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Visualizza lo stato globale di inventario e giacenze.	Nessuno	200 (OK)
/admin/inventory			
	Descrizione	Parametri Richiesti	Codici HTTP
PUT	Aggiorna la quantità di giacenza di un articolo in una specifica filiale.	{article_id, location_id, quantity}	200 (OK) 404 (Not Found)
/admin/internal/allocate			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Effettua l'allocazione logistica prelevando gli articoli dalle filiali.	{order_id, items}	200 (OK) 400 (Bad Request)

Tabella 2.4: Inventory Service - routes/admin.py



Order Service

/client/order			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Crea e piazza un nuovo ordine a nome dell'utente specificato.	{user_id, items}	201 (Created) 400 (Bad Request)
/client/order/<id>			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Traccia lo stato di uno specifico ordine in base all'ID fornito.	Nessuno (id nell'URL)	200 (OK) 404 (Not Found)
/client/order/user/<id>			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Ottiene lo storico di tutti gli ordini completati e falliti dell'utente.	Nessuno (id nell'URL)	200 (OK)

Tabella 2.5: Order Service - routes/client.py

/admin/orders			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Restituisce tutti gli ordini registrati nel sistema.	Nessuno	200 (OK)
/admin/logistics			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Visualizza l'uso corrente e totale dei veicoli logistici per le filiali.	Nessuno	200 (OK)
/admin/logistics			
	Descrizione	Parametri Richiesti	Codici HTTP
PUT	Aggiorna i veicoli logistici totali di una determinata filiale.	{location_id, total_vehicles}	200 (OK) 404 (Not Found)

Tabella 2.6: Order Service - routes/admin.py